

# Personalized Networking Assistant

## Project Description:

The Personalized Networking Assistant is an AI-powered web application designed to help professionals, students, and job seekers build meaningful conversations during networking events. The application leverages Natural Language Processing (NLP) and transformer-based AI models to analyze event descriptions, understand user interests, and generate personalized conversation starters that make networking more engaging and effective.

The system integrates DistilBERT for event theme extraction, GPT-2 Small for conversation generation, and the Wikipedia API for real-time fact verification. The application is built using FastAPI as the backend framework and Streamlit as the frontend interface, providing users with an interactive and responsive experience. Additionally, conversation history and user feedback are stored locally in JSON files, allowing users to revisit previous interactions and improve future recommendations.

The Personalized Networking Assistant reduces the difficulty of initiating conversations at professional events while helping users gain confidence through AI-assisted networking suggestions.

## Scenarios

### Scenario 1: Professional Networking Event

A user plans to attend an Artificial Intelligence conference.

The user provides:

- Event Description
- User Interests
- Professional Background

The system:

- Extracts event themes using DistilBERT.
- Generates three personalized networking conversation starters using GPT-2.
- Stores the generated conversation history automatically.

### Scenario 2: Fact Verification

A user wants to verify information about Blockchain in Healthcare before attending an event.

The user enters the topic.

The application:

- Queries the Wikipedia API.
- Retrieves a concise summary.
- Displays verified information for quick learning.

## Scenario 3: Conversation Review

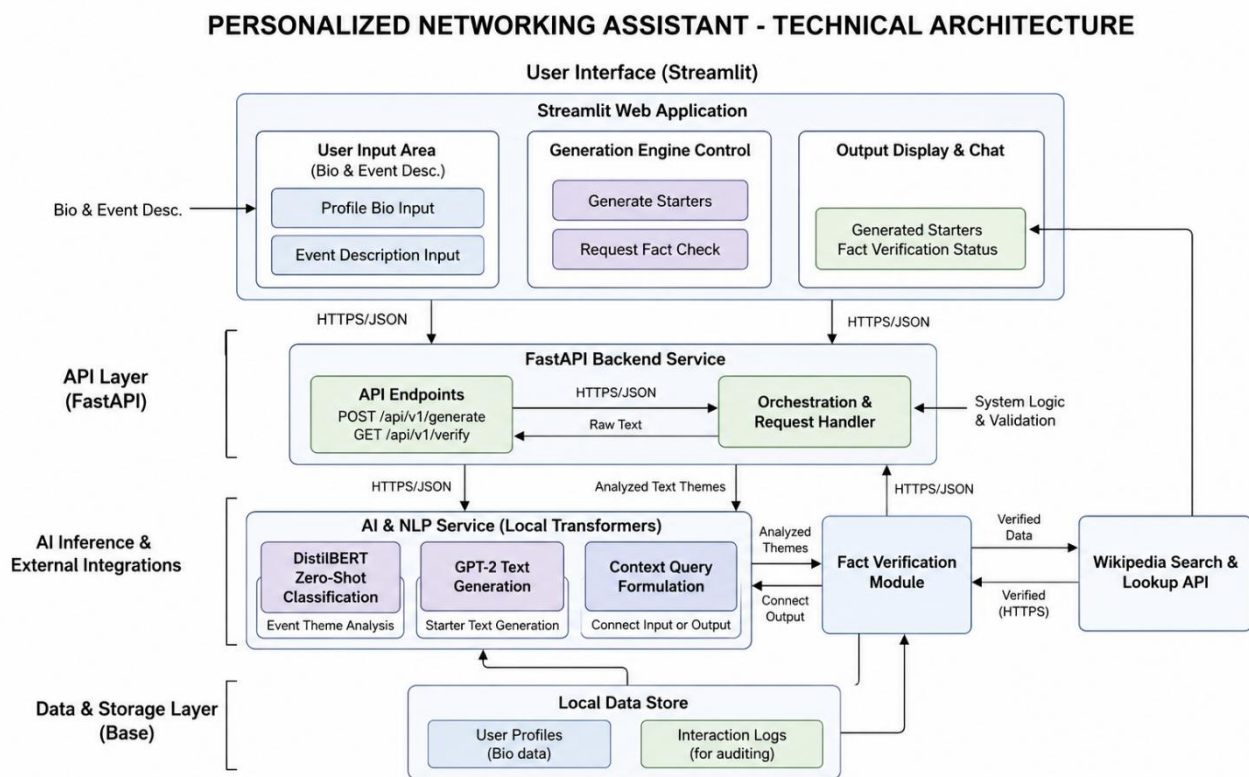
A user wants to review previous networking conversations.

The application displays:

- Event Description
- Extracted Themes
- Generated Suggestions
- Timestamp

allowing users to revisit previous networking sessions.

## Technical Architecture:



## Description :

The Personalized Networking Assistant follows a modular three-tier architecture consisting of:

- Streamlit Frontend
- FastAPI Backend
- AI Services Layer

The frontend collects event descriptions and user interests from users. FastAPI exposes REST APIs that validate incoming requests using Pydantic models and coordinate AI services.

The backend communicates with multiple service modules including:

- Event Analyzer
- Topic Generator
- Fact Checker
- History Logger
- Feedback Logger

The Event Analyzer extracts event themes using DistilBERT Zero-Shot Classification.

## **Pre-requisites:**

Before implementing the application, the following technologies and tools are required:

### **Python Programming**

Python 3.11+

<https://www.python.org/>

### **FastAPI**

<https://fastapi.tiangolo.com/>

### **Streamlit**

<https://streamlit.io/>

### **Hugging Face Transformers**

<https://huggingface.co/docs/transformers>

### **Wikipedia API**

<https://pypi.org/project/wikipedia/>

### **Git**

<https://git-scm.com/>

### **Visual Studio Code**

<https://code.visualstudio.com/>

### **Virtual Environment (venv)**

<https://docs.python.org/3/library/venv.html>

## **Project Workflow:**

### **Milestone 1 : Model Selection and Architecture**

#### **Activity 1.1 Research and Select AI Models**

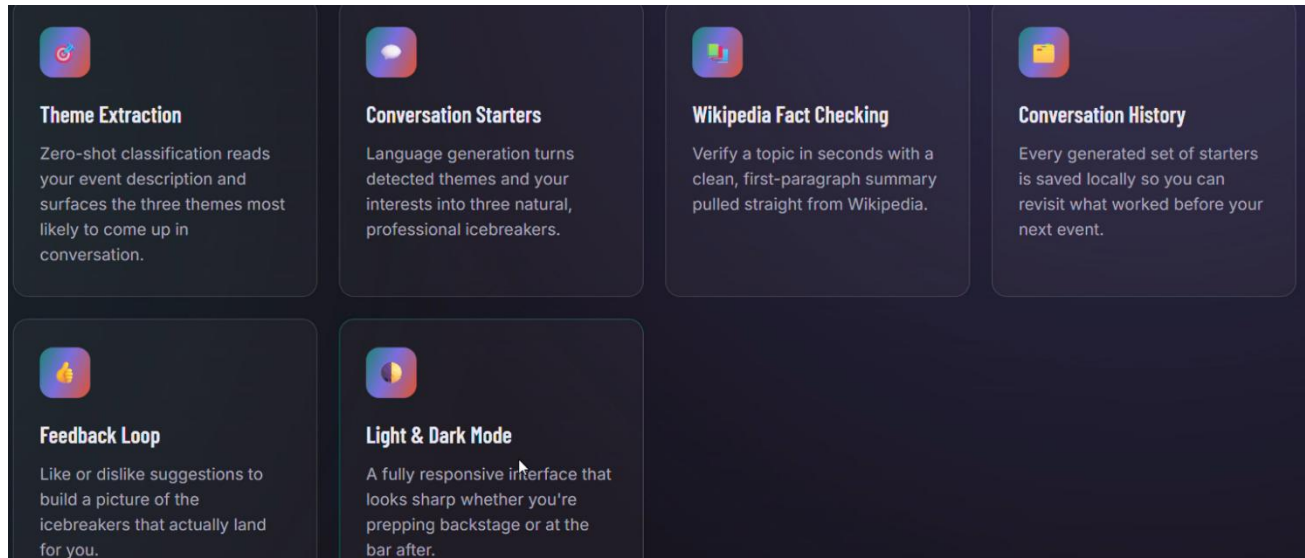
Research various transformer models suitable for Natural Language Processing.

Evaluate:

- DistilBERT
- BERT
- RoBERTa
- GPT-2

Select:

- DistilBERT for Theme Extraction
- GPT-2 Small for Conversation Generation



## Activity 1.2 Define Application Architecture

Design the complete system architecture consisting of:

- Streamlit Frontend
- FastAPI Backend
- AI Service Layer
- JSON Storage

Workflow

User Input

↓

FastAPI API

↓

Event Analyzer

↓

Topic Generator



History Logger



Frontend Results

### Activity 1.3 Environment Setup

Create a virtual environment.

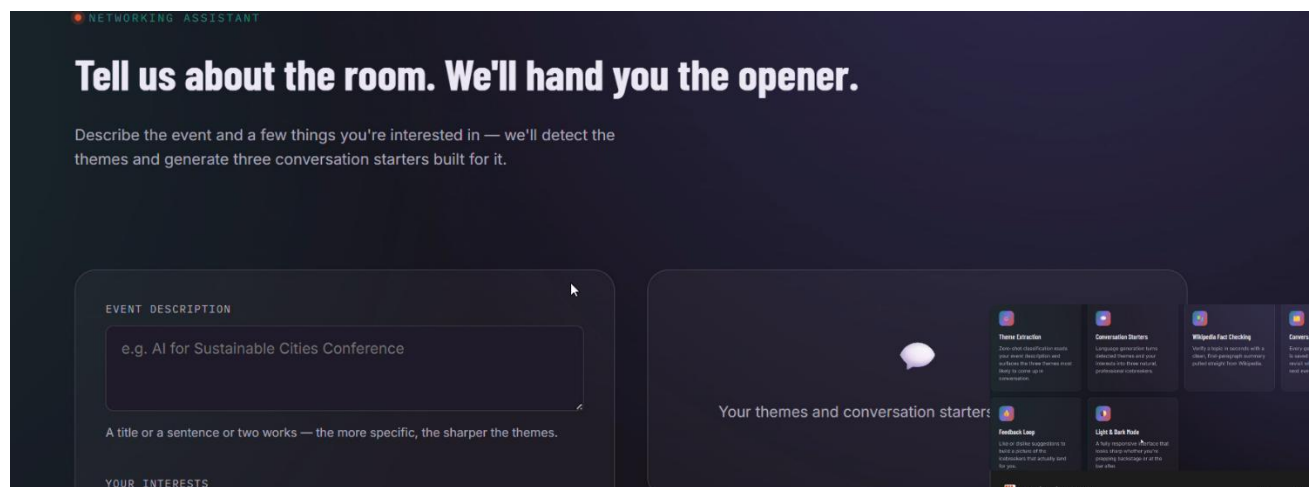
Install project dependencies.

Configure:

- FastAPI
- Streamlit
- Transformers
- Torch
- Requests
- Pytest
- HTTPX

Verify installation of all required packages.

## Milestone 2 : Core Functionalities Development



### Activity 2.1 Data Schema Definition

Create Pydantic models for:

- Conversation Request
- Conversation Response
- Event Request
- Event Response
- Fact Check Request
- Fact Check Response

These schemas validate every API request automatically.

### **Activity 2.2 Event Analyzer Service**

Implement the Event Analyzer using Hugging Face Zero-Shot Classification.

Responsibilities:

- Load DistilBERT model
- Analyze event description
- Extract top three themes
- Return ranked themes

### **Activity 2.3 Topic Generator Service**

Develop GPT-2 based conversation generator.

Responsibilities:

- Accept themes
- Accept user interests
- Generate personalized networking conversation starters
- Return three clean suggestions

### **Activity 2.4 Fact Checker Service**

Implement Wikipedia Fact Verification.

Responsibilities:

- Receive search topic
- Query Wikipedia API
- Return concise summary
- Handle errors gracefully

### **Activity 2.5 History Logger**

Maintain conversation history.

Store:

- Event Description
- User Interests
- Themes
- Suggestions
- Timestamp

### **Activity 2.6 Feedback Logger**

Capture user feedback.

Store:

- Suggestion
- Like
- Dislike
- Timestamp

### **Activity 2.7 FastAPI Routes**

Create REST APIs.

POST

/analyze-event

POST

/generate-conversation

POST

/fact-check

GET

/history

GET

/feedback

POST

/feedback

## **Milestone 3 : Main Application Logic**

### **Activity 3.1 FastAPI Application**

Develop main.py

Responsibilities:

- Create FastAPI application
- Register routers
- Configure APIs
- Provide Health Check endpoint

### **Activity 3.2 Routing Architecture**

Create modular routing using APIRouter.

Routes communicate with service modules only.

### **Activity 3.3 Service Layer Design**

Each service performs one responsibility only.

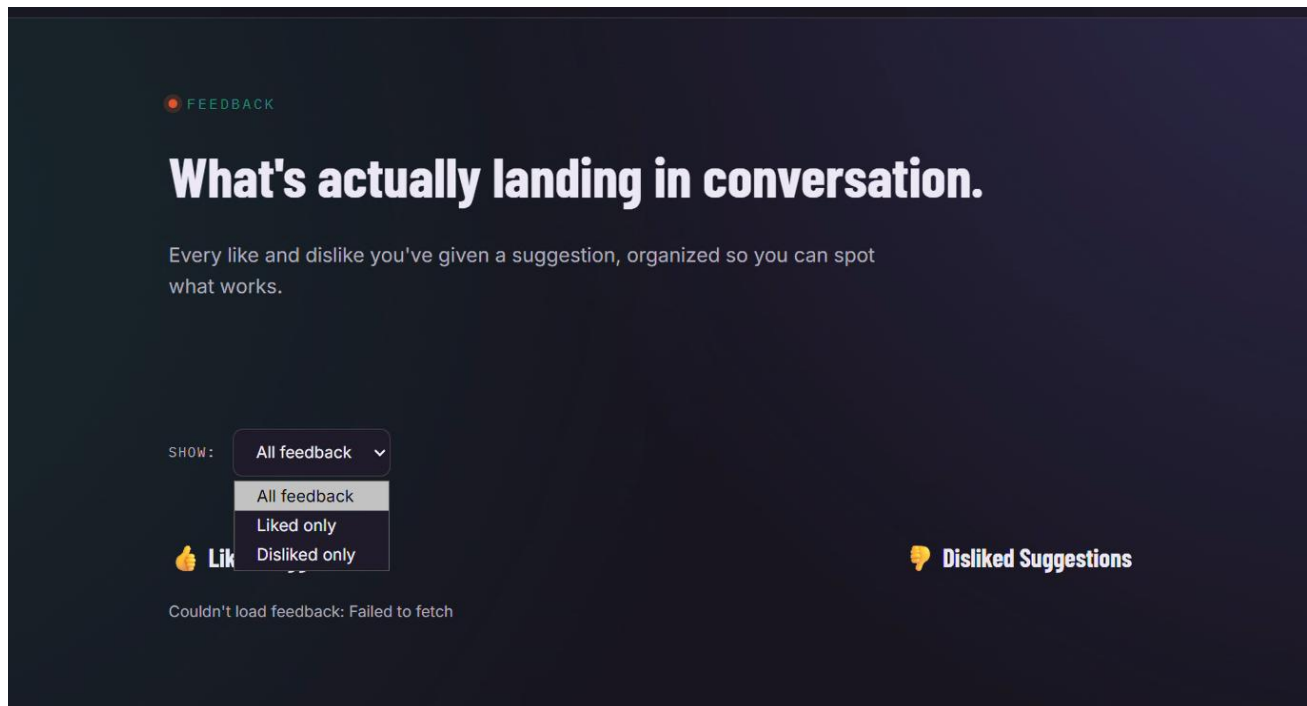
Services include:

- Event Analyzer



- Topic Generator
- Fact Checker
- History Logger
- Feedback Logger

## Milestone 4 : Frontend Development



### Activity 4.1 Streamlit Setup

Configure Streamlit application.

Connect frontend with FastAPI backend.

### Activity 4.2 User Input Interface

Allow users to enter:

- Event Description
- Interests

Generate personalized conversation starters.

### **Activity 4.3 Results Display**

Display:

- Extracted Themes
- Conversation Suggestions

Provide:

- Like Button
- Dislike Button

### **Activity 4.4 Fact Checking**

Allow users to verify topics using Wikipedia.

Display summary inside Streamlit.

### **Activity 4.5 Conversation History**

Display last five conversations.

Each record includes:

- Event
- Themes
- Suggestions
- Timestamp

### **Activity 4.6 Feedback History**

Display previous likes and dislikes.

### **Activity 4.7 Streamlit Session State**

Maintain application state using

`st.session_state`

to preserve results during reruns.

## **Milestone 5 : Testing and Deployment**

### **Activity 5.1 Unit Testing**

Test:

- Event Analyzer
- Topic Generator
- Fact Checker

using pytest.

### **Activity 5.2 API Testing**

Use HTTPX TestClient.

Validate:

- API Responses
- Request Validation
- Error Handling

### **Activity 5.3 Local Deployment**

Run FastAPI

uvicorn app.main:app --reload

Run Streamlit

streamlit run frontend/app.py

Backend

<http://127.0.0.1:8000>

Swagger

<http://127.0.0.1:8000/docs>

Frontend

<http://localhost:8501>

## Conclusion:

The **Personalized Networking Assistant** successfully demonstrates the practical application of Artificial Intelligence and Natural Language Processing in professional networking. By integrating **DistilBERT** for event theme extraction, **GPT-2 Small** for personalized conversation generation, and the **Wikipedia API** for fact verification, the application delivers intelligent, context-aware networking support through an intuitive web interface.

The modular architecture, consisting of **FastAPI**, **Streamlit**, and dedicated AI service modules, ensures maintainability, scalability, and ease of testing. Features such as conversation history, user feedback logging, and structured API design further enhance usability and provide a strong foundation for future development.

The project showcases how AI can simplify networking by helping users initiate meaningful conversations, build confidence in professional interactions, and access reliable information in real time. Future enhancements may include multilingual support, cloud deployment, user authentication, speech-based interaction, and integration with professional networking platforms to further improve user experience and scalability.

